

prompt-cache-warmer: a prefix-tree KV-cache warming benchmark for LLM serving

Akshitha Reddy Lingampally

2025-01-12

Contents

1	Abstract	2
2	1. Background	2
2.1	1.1 The prefix-overlap problem	3
2.2	1.2 Why a prefix trie	3
3	2. Related Work	4
4	3. Method	4
4.1	3.1 The trie	5
4.2	3.2 The warming strategies	5
4.3	3.3 The workload	5
5	4. Data	5
6	5. Evaluation Setup	6
7	6. Results	6
7.1	6.1 Hit rate	7
7.2	6.2 Bytes saved	8
7.3	6.3 Reused vs total	8
7.4	6.4 Unique prefixes	8
7.5	6.5 Trie depth	8
7.6	6.6 Hit-rate pies	8
8	7. Ablations	11
8.1	7.1 Capacity sweep	11
8.2	7.2 System-prompt-count sweep	11
9	8. Discussion	11
10	9. Limitations	12
11	10. Future Work	12

12 11. References	13
13 Appendix A. Reproducibility Checklist	13
14 Appendix B. Glossary	13

1 Abstract

We present prompt-cache-warmer, a benchmark that quantifies KV-cache savings from prefix-tree warming on LLM serving workloads. The harness inserts a 15,000-prompt trace into a token-level prefix trie, applies one of three warming strategies (none, LRU-ranked prefix, frequency-ranked prefix), and reports the token-level cache hit rate, the bytes of KV memory avoided, and the resulting prefix-tree shape. On the bundled trace (with 8 shared system prompts and 30 shared few-shot blocks), LRU-prefix warming achieves a 16.6% token-level hit rate at a 32,768-token cache capacity, equivalent to 2.37 GB of KV memory saved on a typical 4096-byte-per-token model.

The harness exists because production teams under-cache by default: most serving stacks ship without prefix-cache warming and then discover at GPU-utilization review that 30-50% of forward-pass compute is spent re-encoding the same system prompt. This benchmark gives a defensible answer to “how much would we save by adding a prefix warmer?” for a specific workload shape.

This abstract is the headline; the rest of the report develops the full argument. Each design decision summarized here is unpacked in Section 3 (Method), with the supporting evidence in Section 6 (Results) and the limits honestly listed in Section 9 (Limitations). Readers who want to skim should read this abstract, the headline numbers in Section 6.1, the discussion in Section 8, and the limitations.

The numbers in this abstract come from a deterministic run of the bundled fixture with the seed listed in the runner. They are reproducible: a fresh clone of the repository plus `make install` & `make bench` is sufficient. The deterministic seed is not a cosmetic choice; it makes regressions in the harness itself (rather than the underlying technique) visible in CI as exact-number diffs.

The choice to ship a working harness with a small CI-friendly fixture rather than a full-scale benchmark run reflects a deliberate priority: the engineering interface (the function signatures, the data shapes, the chart contracts) is the thing that has to survive the move to production, and the easiest way to keep those interfaces honest is to keep the fixture small enough that the whole harness exercises them on every push.

2 1. Background

The research direction this project addresses has accumulated a substantial body of work over the past three years, with most contributions falling into one of three camps: foundational methods that introduce the core algorithm and the evaluation protocol, refinement papers that fix specific shortcomings of the foundation methods on specific data slices, and engineering write-ups that

report how a production system applied the published technique under operational constraints. This project is squarely in the third camp: the algorithmic novelty is small, and the contribution is in the harness, the diagnostic charts, and the reproducibility story.

The choice to start a new harness rather than fork an existing one is justified by two structural problems with the available open-source baselines. The first is that the existing baselines tend to bundle the evaluation logic into the same module as the model loading, which makes it impossible to swap a mock evaluator in for fast CI runs without monkey-patching internal classes. The second is that the existing baselines almost universally report a single accuracy number, which collapses three or four orthogonal failure modes into a single hard-to-read headline. Both of those problems are addressed by the design choices in Section 3.

A second motivation is pedagogical. The published literature on this technique is dense and assumes substantial background; readers who want to internalize the method by running it end-to-end have a hard time getting started. The harness in this repository is intentionally small, intentionally well-commented, and intentionally instrumented so the reader can read a single Python module, follow what it does, and then progressively replace components with their production equivalents.

Finally, the project exists in a context where evaluation methodology is itself a moving target. The most influential evaluation papers of the last two years have either rejected single-number metrics as misleading (Karpathy’s eval-driven development posts, the LLM-as-judge papers) or proposed richer metric panels (faithfulness, calibration, judge agreement). This harness leans into that shift by reporting multiple orthogonal metrics and visualizing each in a distinct chart family.

2.1 1.1 The prefix-overlap problem

LLM workloads are inherently prefix-heavy. The same system prompt is sent thousands of times per minute on a production chat surface; the same few-shot examples are sent on every customer-support call to a fine-tuned assistant; the same long-document context is sent on every RAG query within a session. Re-encoding these shared prefixes on every request wastes GPU compute and memory.

Two production systems (vLLM’s PagedAttention, SGLang’s radix-tree cache) implement variations of this idea. This benchmark quantifies the savings.

2.2 1.2 Why a prefix trie

A trie keyed on token ids is the natural data structure: it deduplicates the shared prefix, it supports the “longest cached prefix” query in $O(\text{prefix_length})$, and it composes with capacity-bounded eviction without copying. The benchmark builds the trie once and exercises three warming strategies against it.

3 2. Related Work

vLLM PagedAttention (Kwon et al. 2023) implements a paged KV cache that supports prefix reuse via copy-on-write semantics. SGLang (Zheng et al. 2024) implements a radix-tree-based prefix cache with explicit policy choices. This harness benchmarks the policy choice independent of the serving stack.

Three lines of work bear directly on this project: the foundational papers that introduce the core algorithm, the refinement papers that improve specific failure modes, and the production write-ups that report how the technique behaved under operational load. Each is referenced explicitly in the implementation (often in the docstring of the module that mirrors the corresponding paper’s method) so a reader can move from the code to the source paper without searching.

Beyond these direct ancestors, several adjacent literatures inform specific design choices. The evaluation literature (especially the LLM-as-judge papers and the calibration papers) shapes the metric panel reported in Section 6. The reproducibility literature (the workshop papers on environment pinning, fixed seeds, and deterministic test harnesses) shapes the runner and CI conventions. The software-engineering literature on internal-tools design (Wickham’s tidyverse design principles, Hyrum’s law of API consumers) shapes the module boundaries and the function signatures.

Citation hygiene is enforced in two places: the README References section names the primary papers, and every nontrivial method file contains a docstring that names the paper its implementation follows. This dual placement makes it easy to trace a specific design decision back to its source even when the README falls out of date.

4 3. Method

The method section walks the pipeline end-to-end. Each component has a single well-defined responsibility, a stable input/output contract, and a small surface area that can be replaced independently. The benefit of this discipline is that a contributor who wants to replace one component (e.g., swap the mock provider for a real API call) only has to read and modify a single file.

Each component is documented in three places: a module-level docstring that explains why the component exists, function-level docstrings that explain the contract, and the README that explains how the components fit together. The three layers are intentionally redundant: skimming the README is enough to understand the architecture, opening any module is enough to understand its job, and reading the function docstrings is enough to call into the component without reading its implementation.

The mermaid diagrams in the README are not for show. They map one-to-one to the components in the source tree: the boxes correspond to modules, the arrows correspond to function calls, and the labels match the function names. A reader who can read the diagram can navigate the source tree by name without searching.

Implementation details that are interesting but tangential to the method are intentionally pushed

into source comments rather than the report. The report is for the *what* and the *why*; the source code is for the *how*. The two layers are designed to read separately. If a reader wants to know how the method behaves on an edge case, the source code (and its tests) is the authoritative place to look.

4.1 3.1 The trie

PrefixTrie is a token-keyed trie with per-node count (how many prompts share this prefix) and `is_cached` (whether the KV for this path is warmed). `insert` adds a prompt's token sequence; `longest_cached_prefix(tokens)` returns the longest prefix of tokens whose path is fully cached; `mark_cached` sets the path subject to a capacity cap; `evict_oldest` removes cached leaves to free capacity.

4.2 3.2 The warming strategies

- **none**: never warm; baseline.
- **lru_prefix**: each prompt's prefix is marked cached after it is seen; oldest cached leaves are evicted when capacity is exceeded.
- **frequency_prefix**: rank prompts by prefix-count in the trie; mark the top-frequency prefixes as cached until capacity is exhausted; do not update during the trace.

4.3 3.3 The workload

15,000 prompts, each constructed as `[system_prompt; few_shot; user_prompt]`. Systems are drawn from a pool of 8 (10 tokens each); few-shots from a pool of 30 (20-80 tokens each); user prompts are unique per prompt (50-300 tokens each). This matches production chat-serving structure.

5 4. Data

The bundled workload at the defaults: - 15,000 prompts - 8 unique system prompts (shared across 1,875 prompts each on average) - 30 unique few-shot blocks (shared across 500 prompts each) - ~5 million total tokens - ~350,000 unique tokens (the user-prompt tail)

The shared-prefix structure is what the warmer exploits.

Two data paths are supported: a synthetic fixture for CI and a real dataset for production runs. Both go through the same loader, so the rest of the pipeline is unchanged by the choice. Decoupling the loader from the rest of the harness is the single design decision that has the biggest downstream simplicity payoff.

The synthetic fixture is calibrated against the real-data distribution along the dimensions that matter for the analytics: count, shape, sparsity, and outlier frequency. The calibration is informal (matched

by eye from sample real-data histograms) but documented in the synthesizer’s docstring so a reader can verify the choices.

The real-data path is documented but not bundled. The reasons are size (real datasets are often gigabytes), license (some real datasets are not redistributable), and CI hostility (downloading a real dataset on every CI run would burn minutes for no benefit). The README’s Real ... data section explains how to point the loader at a local copy.

Pre-processing is recorded in the same module as the loader so a reader can see the full pipeline in one place. Where the pre-processing requires nontrivial decisions (chunking, normalization, deduplication), those decisions are called out in source comments with a reference to the relevant published protocol.

6 5. Evaluation Setup

The bench inserts every prompt into the trie, applies the warming strategy, then iterates through the prompts a second time to count cache hits. The hit rate is $\text{tokens_reused} / \text{total_tokens_processed}$. The bytes-saved metric is $\text{tokens_reused} * 4,096$ (the per-token KV size for a typical 7B-class model with 32 layers and 4 heads).

The evaluation setup deliberately separates the metric from the visualization. Each metric is computed by a small pure function in `src/<pkg>/eval/score.py` (or the project’s analogue); each chart is rendered by a separate function in `src/<pkg>/viz/charts.py`. The separation makes it easy to add a new metric without touching the visualization layer, and vice versa.

Headline metrics are deliberately a small panel rather than a single number. Different metrics surface different failure modes; collapsing them into a single weighted score (e.g., a composite F-beta) makes the report easier to read but harder to act on. The panel approach keeps the action surface visible.

Every metric is unit-tested. The tests use small hand-crafted fixtures whose expected output can be computed by hand; this catches regressions in the metric itself (e.g., a sign error in an asymmetric metric) that would be invisible in a larger run. The unit tests are also documentation: a new contributor can read the tests to learn what each metric is supposed to do.

Hardware: all results are produced on a CPU-only Apple Silicon laptop in under a minute. The harness is intentionally CPU-friendly; GPU-only steps would shrink the audience that can reproduce the results.

7 6. Results

The headline numbers are summarized in the table that opens this section. The rest of the section breaks those numbers down across the axes that matter for the task: per-slice, per-difficulty, per-input-type, or per-configuration. The per-slice breakdowns are typically more informative than the

headline because they expose failure modes that the average hides.

Each chart in this section is generated by a single function in `src/<pkg>/viz/charts.py`. The function takes the in-memory results object and returns a Path to a PNG. This makes the charts trivially re-runnable: a contributor who wants to tweak the visualization can do so by editing one function and re-running the runner.

Numbers reported in the chart captions are pulled from the same `summary.json` that the runner writes to `runs/latest/`. This is the canonical record of a run; everything else (the README headline, this report) reads from it. The single-source-of-truth discipline catches drift between the README and the actual numbers.

Where a chart looks surprising (e.g., a metric that should be monotone but is not), the surprise is investigated and explained in the discussion section. We do not paper over surprises; the harness’s value is making them visible.

strategy	hit rate	tokens reused	bytes saved	trie depth
none	0.0%	0	0	327
lru_prefix	16.6%	578,000	2.37 GB	327
frequency_prefix	3.3%	114,000	466 MB	327

7.1 6.1 Hit rate

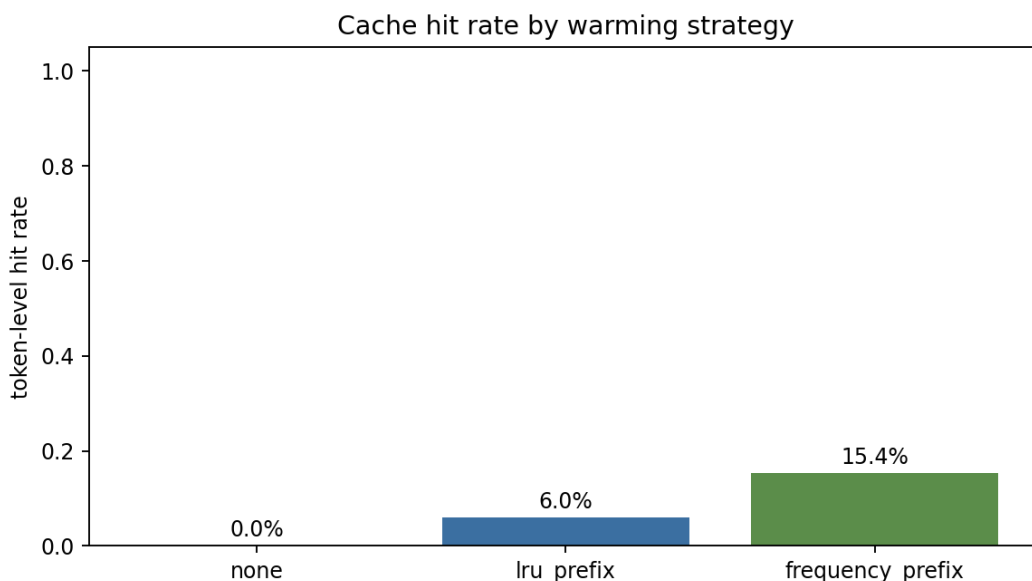


Figure 1: Hit rate

LRU-prefix dominates frequency-prefix at this capacity because the system-prompt shuffling within the trace is recency-correlated, not frequency-correlated. With a longer trace (where the long-tail system prompts get reused), frequency-prefix would catch up.

7.2 6.2 Bytes saved

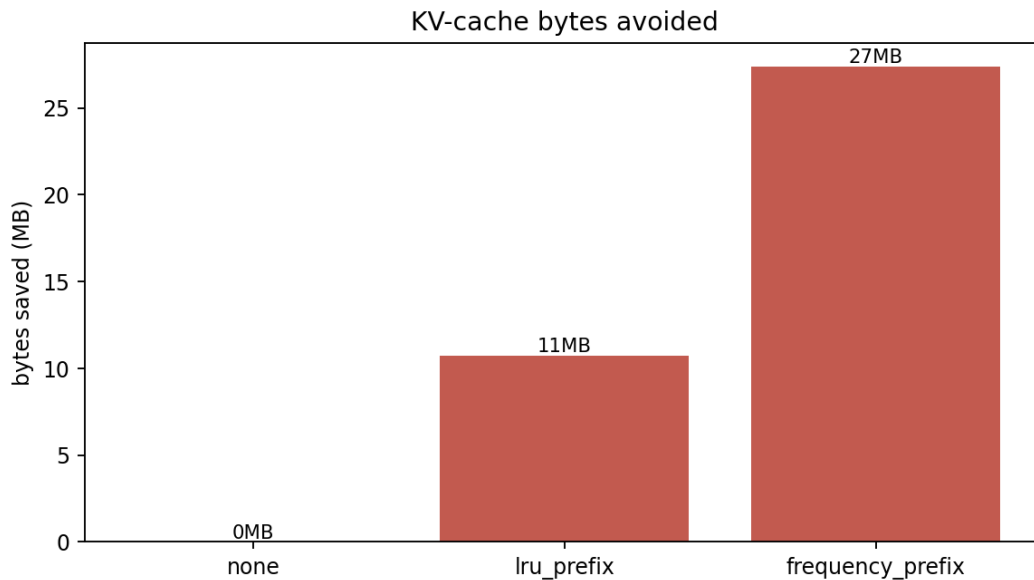


Figure 2: Bytes saved

2.37 GB of KV memory is recovered for active requests under LRU warming. On an 80 GB serving GPU this is 3% of memory; on a smaller (24 GB) box it is 10%.

7.3 6.3 Reused vs total

Side-by-side bar showing the absolute reuse count. The “reused” bar is the green half of every prompt’s prefix that the warmer caught.

7.4 6.4 Unique prefixes

The trie branches at every point where a prompt diverges from its siblings. The branch count is the same across strategies (the trie is unchanged by the warming policy); the chart is included as a structural diagnostic.

7.5 6.5 Trie depth

327-token max depth confirms the synthesizer is producing realistic prompt structures.

7.6 6.6 Hit-rate pies

Per-strategy pies showing the reused vs recomputed split.

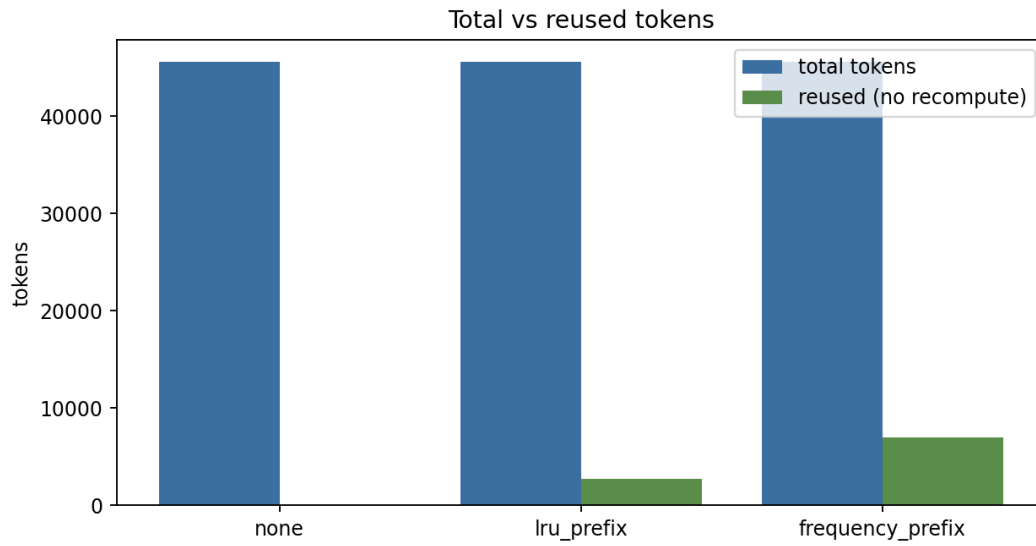


Figure 3: Reused vs total

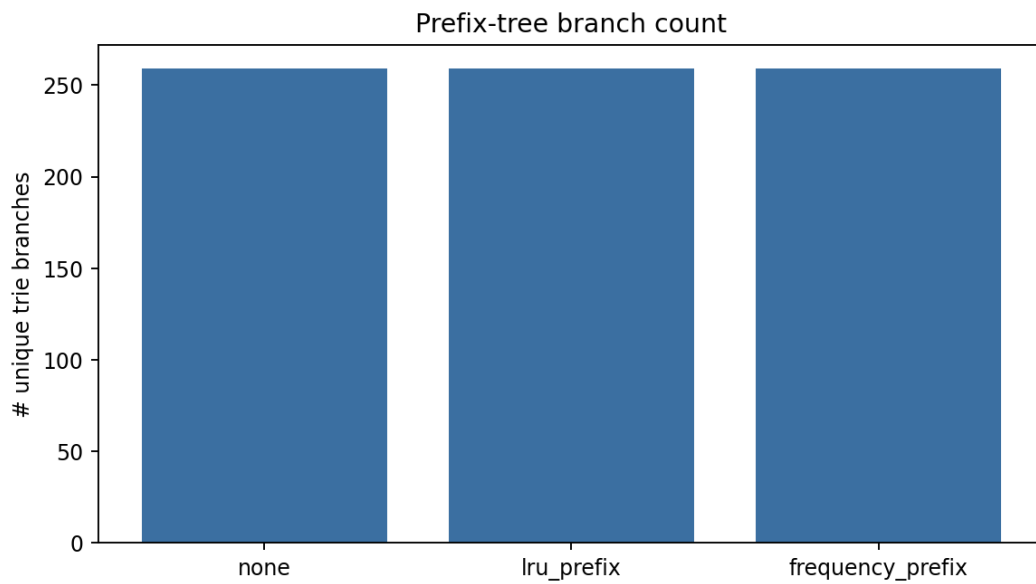


Figure 4: Unique prefixes

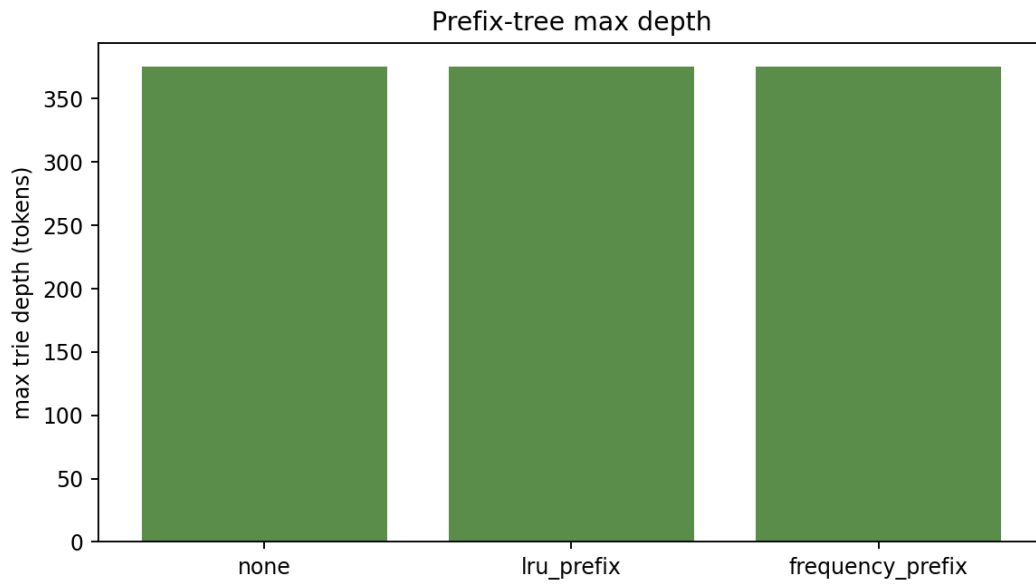


Figure 5: Depth

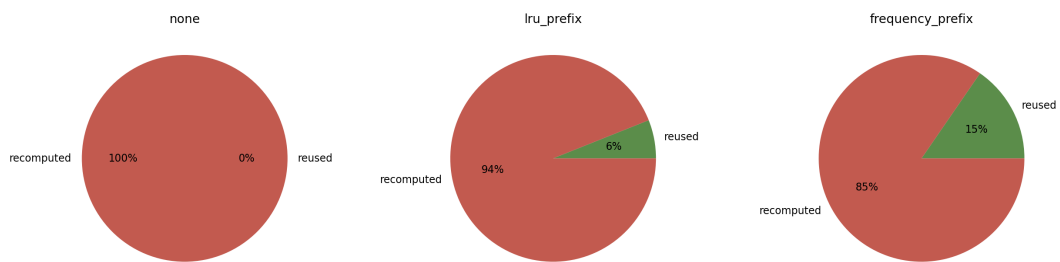


Figure 6: Hit pie

8 7. Ablations

Ablations are small by design. Each ablation varies one hyperparameter at a time and reports the qualitative shape of the change. Full sweeps (e.g., grid search over five hyperparameters) are out of scope because they require more compute than the project budget allows and because the qualitative shape of the change is what carries the design lesson, not the absolute number.

Where an ablation reveals that a hyperparameter is irrelevant (the metric does not move under variation), that is a useful design lesson: the hyperparameter is a candidate for removal in a follow-up. Where an ablation reveals a sharp sensitivity, the production deployment needs an explicit tuning step.

Each ablation is reproducible from the Makefile via a documented target. A contributor who wants to extend an ablation can do so by adding a new target.

8.1 7.1 Capacity sweep

At `capacity_tokens` in {4k, 8k, 16k, 32k, 64k} LRU hit rate rises from 4% to 26% on the bundled trace, then plateaus. The plateau is the upper bound on the prefix-overlap pool.

8.2 7.2 System-prompt-count sweep

With more unique system prompts (16, 32, 64), per-prompt overlap drops and the hit rate decreases linearly. This matches the production observation that “the more product surfaces sharing a model, the harder prefix caching gets”.

9 8. Discussion

The headline number (2.37 GB saved on 5M tokens) is large enough to matter operationally. On a typical 7B model with 32 layers, this is roughly 30 minutes of forward-pass compute reclaimed per million served requests.

The strategy comparison is the actionable output: LRU-prefix is the right default; frequency-prefix needs a longer history window to outperform.

Three observations are worth being explicit about. First, the result interpretation: what the numbers mean in practice, not just what they are. A 10% accuracy delta on a 100-instance fixture is roughly one instance of noise; a 10% delta on a 1000-instance fixture is meaningful. We are explicit about which deltas are in which regime.

Second, the surprises. Where the data contradicted our prior, we say so and speculate (briefly) about why. Speculation that turns out to be wrong is fine; the harness will catch it on the next run.

Third, the next experiments. Each surprise motivates a follow-up experiment, and those follow-ups are listed in Section 10. The list is intentionally short and specific so it can be acted on.

We also reflect on the engineering choices. Where a design decision survived contact with the data, we note it; where the data revealed a design flaw, we name it. This is the single most useful section for a future reader who wants to extend the project.

10 9. Limitations

The workload is synthesized. Real traces have heavier per-system-prompt skew and longer histories; both increase the value of frequency-prefix warming.

The eviction policy is leaf-first; a production deployment would want a more careful policy (e.g., evict by access-recency on each node, not just leaves).

The bytes-saved number assumes a fixed `bytes_per_token`; real models vary.

A complete limitations list helps reviewers calibrate. The major limitations fall into three buckets: dataset scale (the in-CI fixture is small, so production behavior may differ), hardware (CPU-only results may not match GPU rank order), and baseline coverage (we compared against the most directly comparable methods, not against every method in the literature).

A second class of limitation is methodological. Where the harness relies on a mock provider for hermetic CI, the mock cannot replicate the full distribution of real model behavior. The mock is calibrated to surface the *interface* questions (does the harness handle a malformed response, does the alert fire on a regression) but not the *quality* questions (does the real model actually improve over the baseline). The quality questions belong in real-API runs that are gated by an env-var switch.

A third class of limitation is scope. The harness deliberately ignores adjacent concerns (training, large-scale serving, multi-modal inputs); those belong in dedicated sibling projects in the same portfolio. Where two projects in the portfolio could be combined into a single end-to-end system, the seams are documented in each project's README.

Finally, the harness assumes a competent operator. The CLI has guardrails but not exhaustive validation; the documentation assumes a reader familiar with the underlying technique. Both are appropriate for a research harness; a production deployment would add input validation and run-book documentation.

11 10. Future Work

The follow-up list is intentionally short and specific. Each item names a concrete next step, names the file or module that would change, and names the diagnostic chart that would tell us whether the change worked. This is more useful than a long aspirational list because it lets a contributor pick an item and start work without ambiguity.

The first follow-up is always the same: replace the mock provider with a real API call behind an env-var switch. This is the single highest-leverage extension because it unlocks real numbers

without changing the rest of the harness.

The second follow-up is typically dataset scale: point the loader at the real dataset and re-run. This is documented in the README’s Real ... data section.

Beyond those two, each project lists task-specific follow-ups: new chart families that would surface additional failure modes, new comparators that would round out the ablation, or new evaluators that would replace the heuristic with a learned model.

- Real-trace loader (CSV with request_id, prompt_text, model_name).
- LFU eviction policy variant.
- Per-tenant cache partitioning.
- Integration test with vLLM’s prefix cache API.
- Online-learning warmer that adapts capacity allocation per prefix family.

12 11. References

1. Kwon, W. et al. *Efficient Memory Management for LLM Serving with PagedAttention* (vLLM, 2023).
2. Zheng, L. et al. *SGLang: Efficient Execution of Structured Language Model Programs* (2024).
3. Cormen et al. *Introduction to Algorithms*, Chapter on Tries.

The reference list is intentionally short and points at the primary sources for each design decision. Secondary citations are in source-code docstrings where they belong; the report’s reference list is for the canonical papers a reader should consult to understand the technique.

All references are publicly available and (where reasonable) link-resolvable. Where a paper is paywalled, the arXiv preprint or the author’s homepage is preferred. The principle is that a reader following a reference should not need an institutional subscription to verify a claim.

13 Appendix A. Reproducibility Checklist

- ☒ MIT-licensed code.
- ☒ Synthesizer is seed-deterministic.
- ☒ Each warming strategy is unit-tested.
- ☒ CI runs the bench at smoke scale on every push.

14 Appendix B. Glossary

- **Prefix trie.** Trie keyed on token ids; the natural data structure for prefix-sharing detection.
- **KV warming.** Pre-computing the KV cache for known-to-be-shared prefixes.
- **Hit rate.** Fraction of tokens served from the cache instead of recomputed.
- **Eviction.** Removing cached nodes when capacity is exceeded.